

Scene Creation and Management

Learning Objectives

- Understand the concept of scenes in Unity.
- Learn how to create and organize scenes effectively.
- Grasp scene management techniques for complex projects.

What are Scenes in Unity?

Scenes are where you work with content in Unity.
They are assets that contain all or part of a game or application. [8]

A scene is the term Unity uses to describe what you might already know as a level. As you develop a Unity project, each collection of objects and behaviors should be its own scene. Therefore, if you were building a game with a snow level and a jungle level, those would be separate scenes. [3]

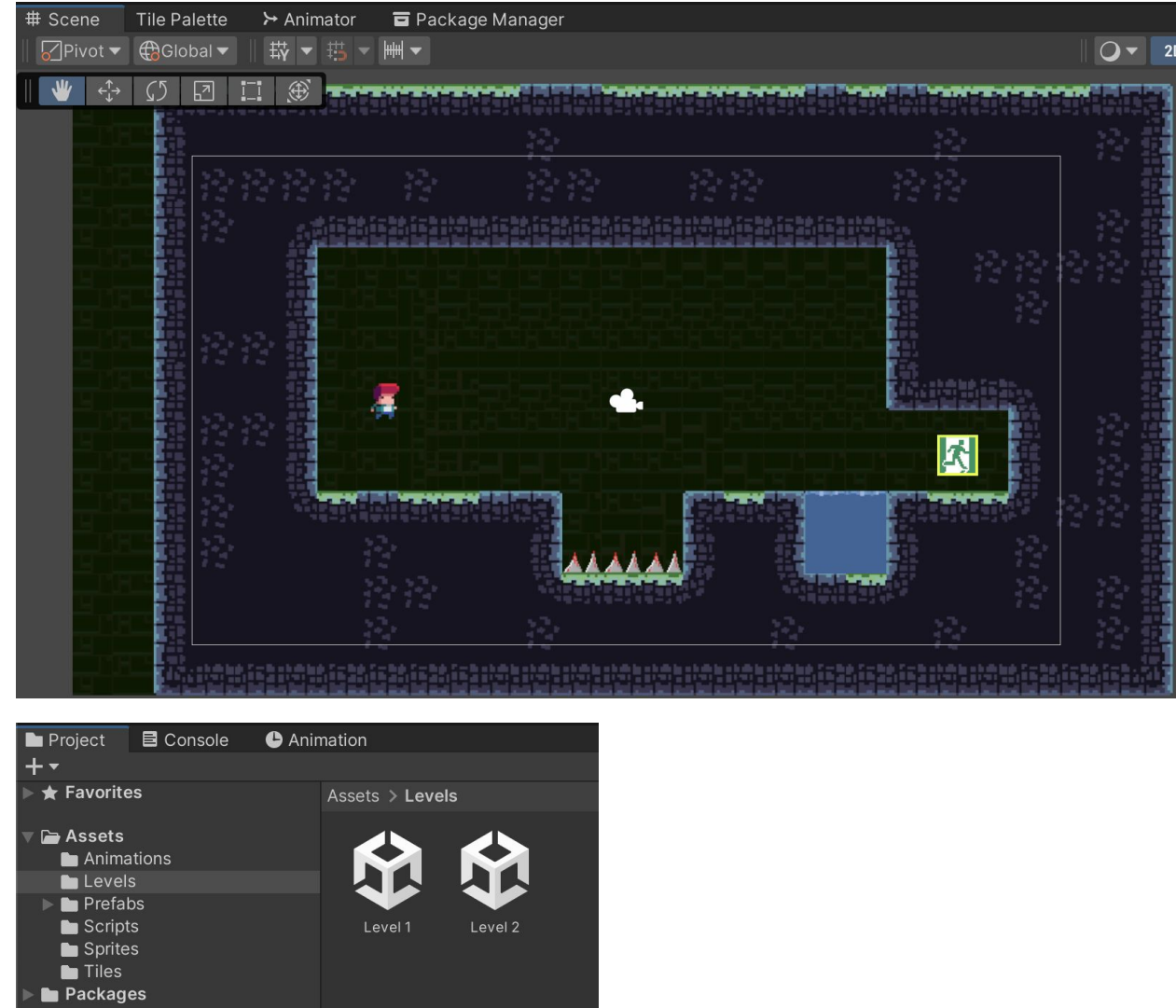
Although it is certainly possible to build large and complex games in this way, it is generally much easier to use multiple scenes. The idea behind a scene is that it is a self-contained collection of game objects. [3]



Scene Creation Basics

1. Open Unity and click on "File."
2. Select "New Scene" to create a new environment.
3. Name your scene and save it in your preferred location.

Double click on scene icon on project window to open a scene



Scene – Interface Overview

Essential Interface Elements

- **Scene View:** Provides a visual representation of the scene. Here, you can position, rotate, and scale objects.
- **Hierarchy Panel:** Lists all GameObjects in the scene, allowing you to organize and manage them hierarchically.
- **Inspector Window:** Displays properties of selected GameObjects, enabling modification of their attributes.
- **Toolbar:** Houses tools for navigating and manipulating the Scene view, including Move, Rotate, and Scale.

Working with Scenes

Key Points

- Scene creation involves organizing and arranging GameObjects.
- The Scene view enables visual editing, while the Hierarchy panel offers a structured view of GameObjects.
- Understanding interface elements is crucial for efficient scene manipulation and design.

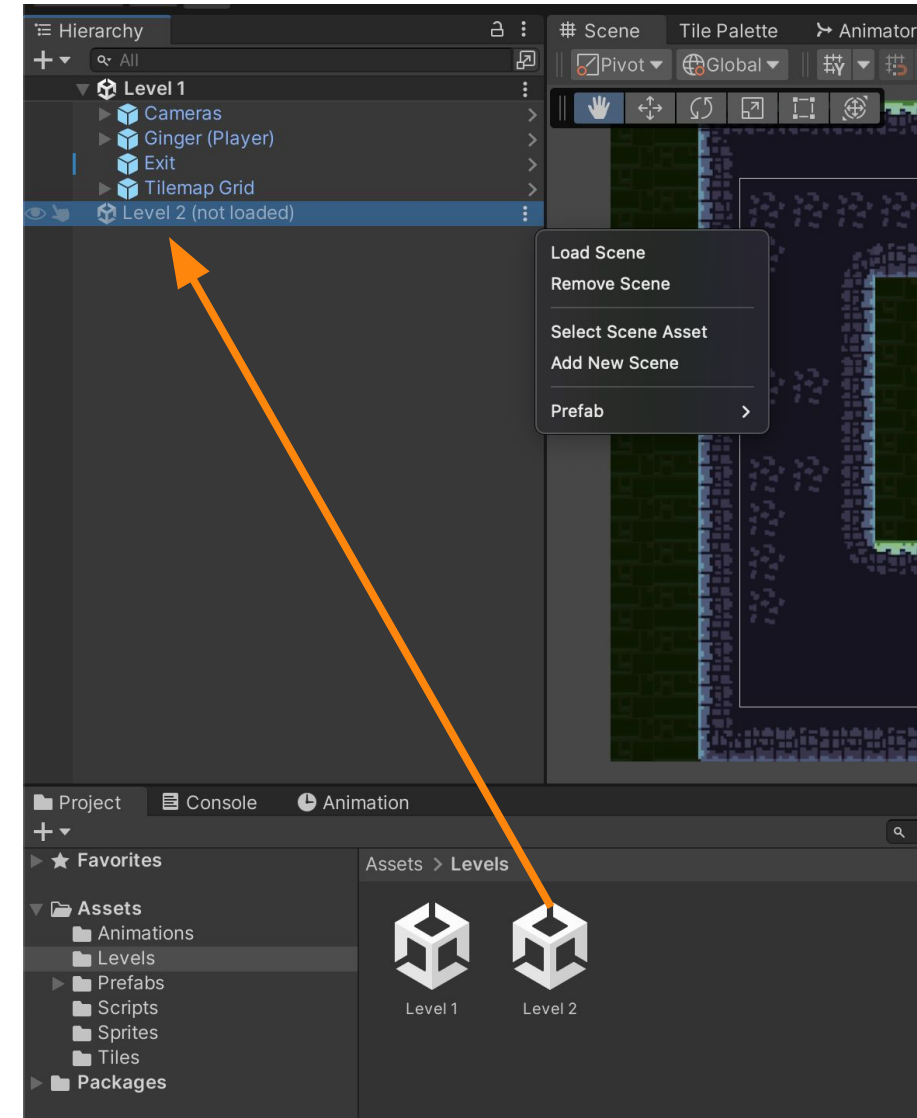
Navigating Between Scenes

UnityEngine.SceneManagement provides functionalities for managing scenes, including switching between them, adding/removing scenes, and controlling their states

Loading Scenes: Use SceneManager.LoadScene() to load a new scene, either by scene name or by index in the build settings.

Unloading Scenes: Use SceneManager.UnloadScene() to remove unnecessary scenes when they're no longer needed.

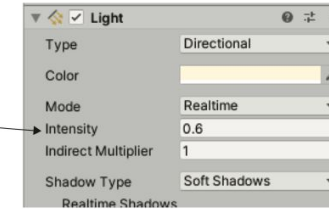
Scene Switching: Use SceneManager.LoadScene() to switch from the current scene to another.



Scene Lighting and Camera Setup

Types of lights: You can create several types of light sources, defined by how and where they project light rays. The three main types are **point**, **spot**, and **directional**.

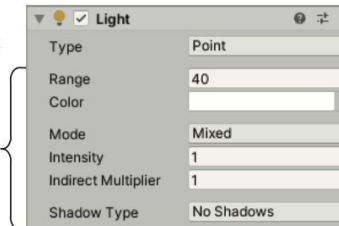
Here is where you control the light's brightness, from 0 for completely dark.



The remaining settings don't need to be adjusted right now. These settings include the color of the light, shadows cast by the light, and even a silhouette projection (think of the Bat signal).

Figure 2.8 Directional light settings in the Inspector

Other than Range, the settings for point lights are the same as for directional lights.



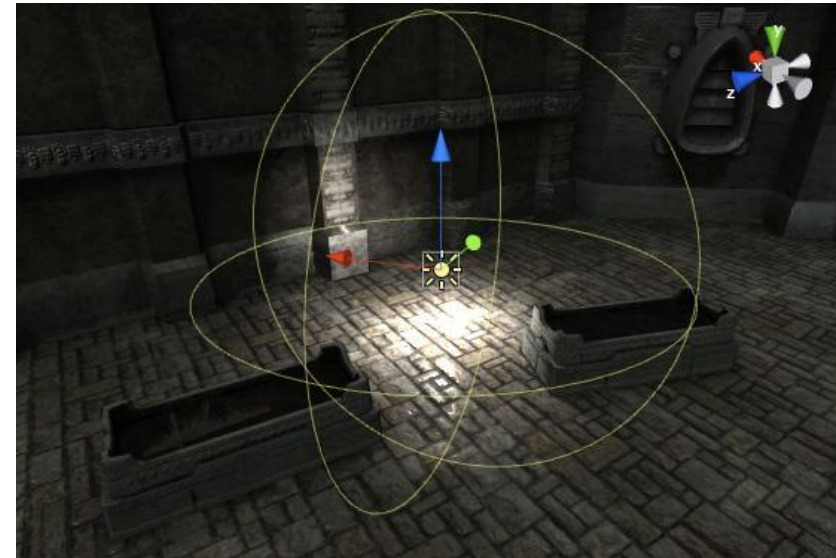
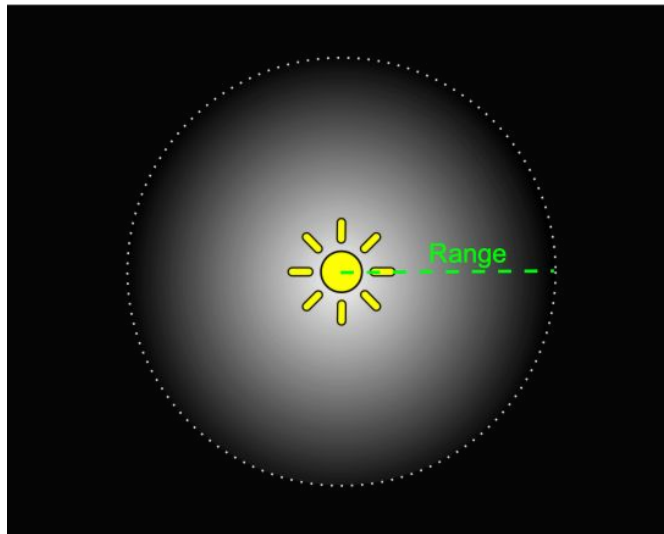
Here is where you control light range, with the same units as position and scale.

(If you see an error about `realtime` not supported, just ignore it or switch Mode to Mixed.)

Scene Lighting and Camera Setup

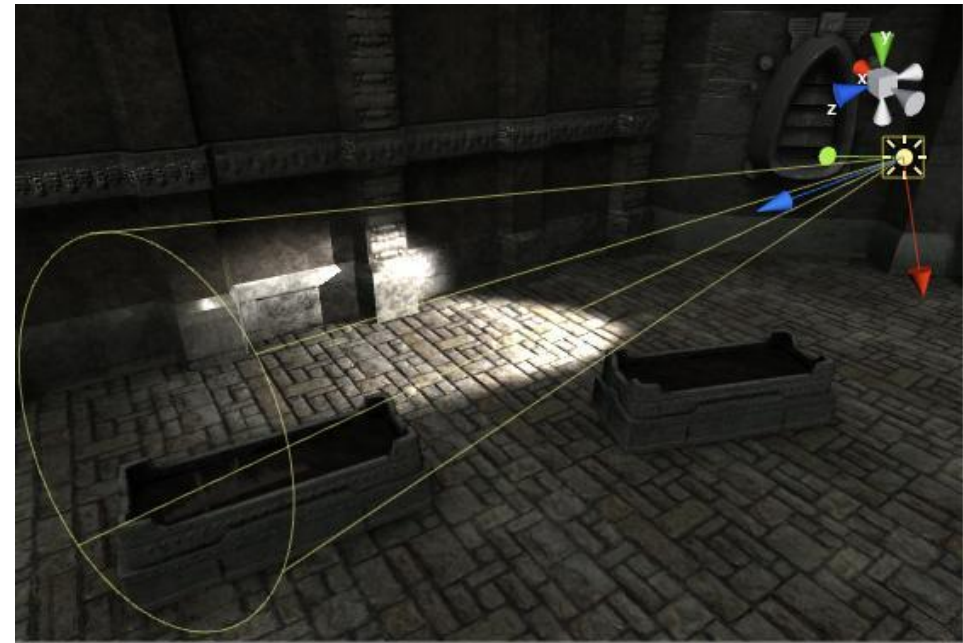
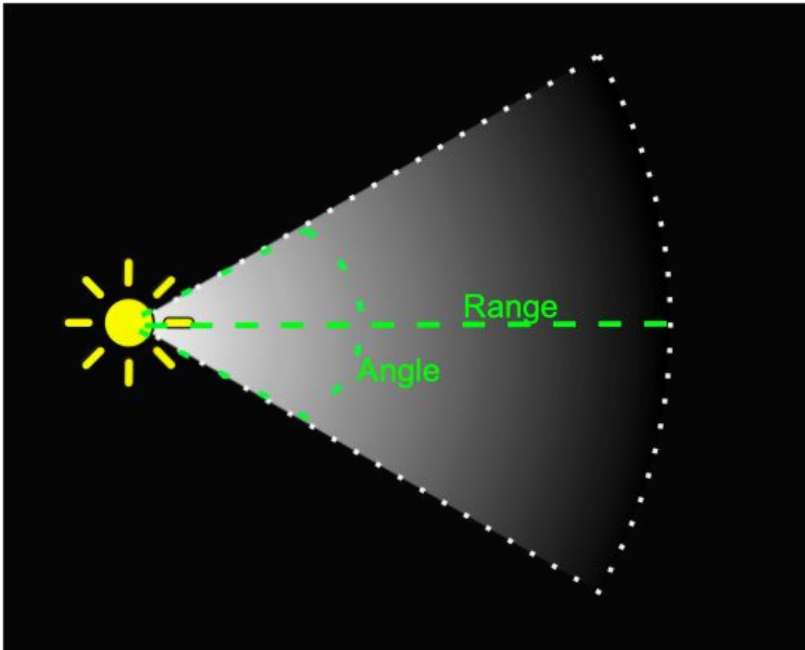
In point lights, all the light rays originate from a single point and project out in all directions, like a light bulb in the real world. The light is brighter up close because the light rays are bunched up.

Point lights are useful for simulating lamps and other local sources of light in a scene. You can also use them to make a spark or explosion illuminate its surroundings in a convincing way.



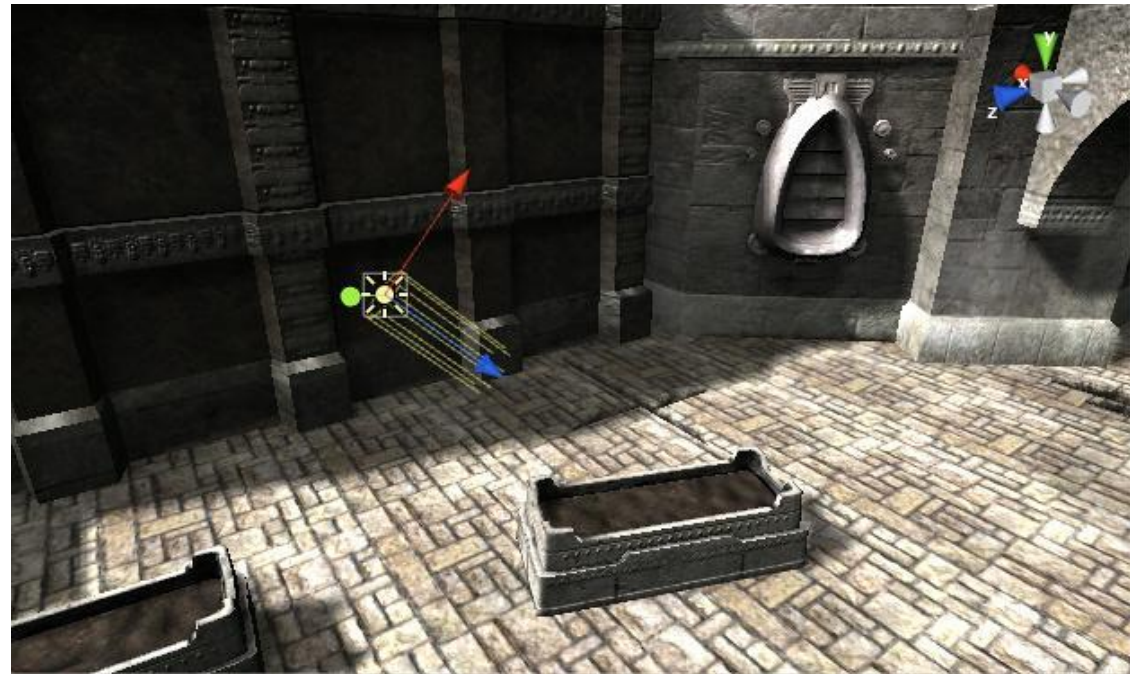
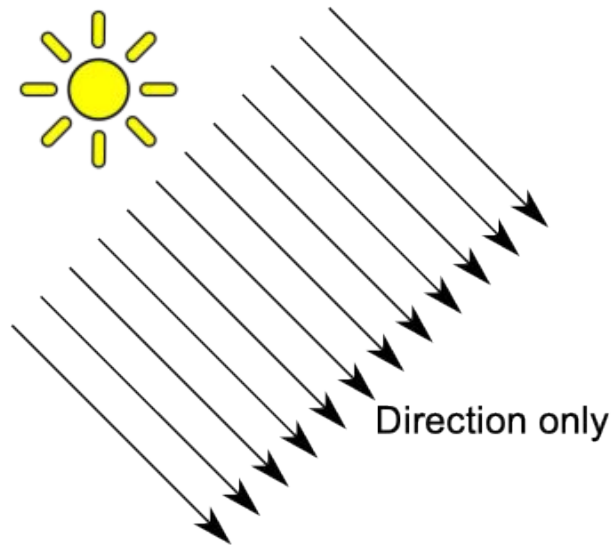
Scene Lighting and Camera Setup

In **spot lights**, all the light rays originate from a single point but project out in only a limited cone. No spot lights are used in the current project, but these lights are commonly used to highlight parts of a level.



Scene Lighting and Camera Setup

In **directional lights**, all the light rays are parallel and project evenly, lighting everything in the scene the same way. This is like the sun in the real world



Scene Saving and Loading

Scene saving and loading in Unity involves preserving the state of a scene so that it can be restored or reloaded at later time. This process is crucial for games where players need to save their progress or return to specific points within the game.

Scene Saving and Loading

Data Collection: Identify and gather relevant data that needs to be saved

Serialization: Convert this gathered data into a serializable format (JSON, XML, binary, etc.) using Unity's serialization techniques or custom serialization methods.

Storage Management: Save the serialized data to a storage medium to persist across game sessions.

Loading Process: Retrieve the saved data from storage when needed, such as when a player wants to resume a saved game or reload a specific scene state

Scene Saving and Loading

Scene Reconstruction: Use the deserialized data to reconstruct the scene's previous state, setting object positions, restoring player progress, and applying any other saved information.

Implementation in Unity: Unity offers PlayerPrefs for simple data storage, and developers can also create custom scripts to manage more complex data saving/loading.

ScriptableObjects, PlayerPrefs, or external files are commonly used to save game settings, player progress, or scene states.

Build Settings and Player Configuration

Build Settings

The Build Settings window contains the terms under which the game is built. It is here that you specify the platform the game will be built under as well as the various scenes in the game. You have seen this dialog once before, but now you should take a closer look at it.

To open the Build Settings dialog, click **File > Build Settings**. Once the Build Settings dialog opens, you can change and configure your game as you want.

Build Settings

Adding scenes to Build Settings in Unity is a crucial step when preparing to build your game. This process determines which scenes you've created in the Editor will be included when you publish your game.

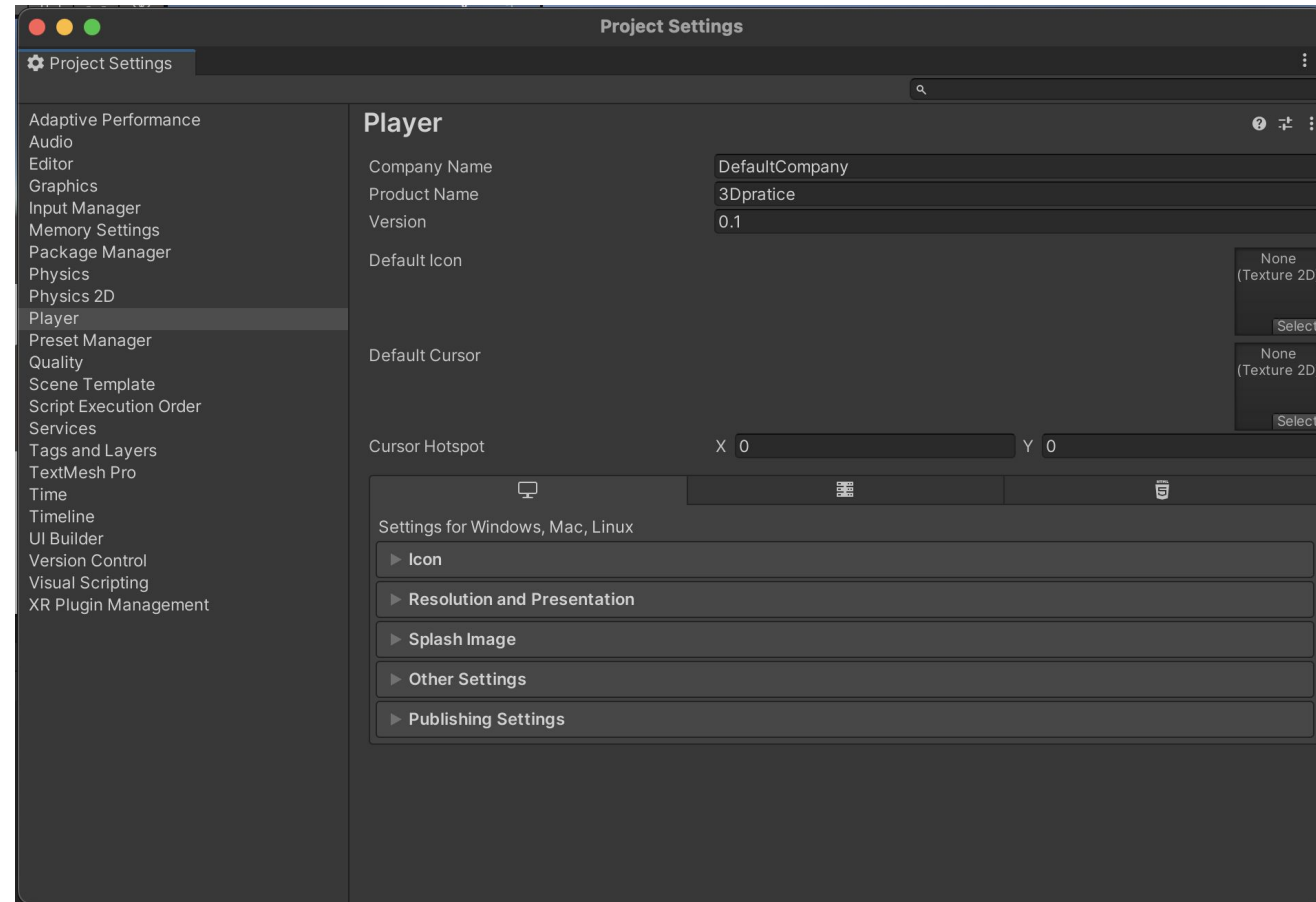
Add Scenes

Drag and drop scenes from the Project Window into the "Scenes In Build" section or click on "Add Open Scenes" to automatically add scenes that are currently open in the Editor.

The scene at the top of the list will be the initial scene loaded upon starting the game. Adding scenes to Build Settings not only determines the game's startup sequence but is also a crucial step in defining the necessary content for the final build, optimizing performance, and enhancing the overall player experience.

Player Configuration

In Unity, when exporting a build you are effectively placing your content into what is known as the Unity player. As a standalone build, the player is part of the packaged executable PC or Mac game.



Player Configuration

In **Player Settings**, you can specify certain elements, such as resolution, icons and rendering settings, for the player to use.

The **Player Settings** in Unity are divided into two core separations—**Cross-Platform Settings** and **Per Platform Settings**.

Player Configuration

Cross-Platform Settings

The cross-platform settings are required for all builds, and simply ask you to provide a **Product Name** for the project, as well as a **Company Name** and **Default Icon**. This icon can be overridden by the per platform settings where required, but otherwise it is generally acceptable to provide a higher resolution image for this setting (such as the 128x128 image provided for this book), and allow Unity to scale where appropriate. The other option is to provide an icon you have designed yourself for each icon resolution within the settings for individual platforms – this approach ensures total control for you but it is best to try the former approach in case it *just works* for you.

Player Configuration

Per-Platform Settings

These further settings for your chosen platform are divided up into four categories:

- Resolution and Presentation
- Icon
- Splash Image
- Other Settings

As both the Web Player and Standalone builds feature these categories, let's look at each of them for both platforms.

Practical Exercise - Scene Creation

1. Change name of current scene to `_Scene_0`
2. Create new scene for Airplane game, called `_Scene_1`.
3. Create Prefab for Airplane GameObject in `_Scene_0`
4. Drag and drop Airplane Prefab to new scene and change its name to Player.

Advance practice

1. Create the level exit trigger, and switch to `_Scene_1` when the airplane go through the trigger cube.

Practice 1 – Creating a New Scene

Creating a New Scene

1. Open Unity and create a new project (if you don't have one already).
2. Go to File > New Scene or use the shortcut Ctrl + N (Cmd + N on Mac) to create a new scene.

Practice 1 – Understanding the Interface

Familiarize yourself with the Unity interface:

- Scene view: Manipulate GameObjects, navigate the scene.
- Hierarchy panel: List of GameObjects in the scene.
- Game view: Preview how the scene looks in play mode.
- Inspector: Details of selected GameObjects.
- Toolbar: Access various tools for editing the scene.

Practice 1 – Manipulating GameObjects

- In the Hierarchy panel, create a few GameObjects (e.g., cubes, spheres, or sprites) by right-clicking and choosing "Create Empty" or using GameObject > 3D Object.
- Explore the Transform tools (Move, Rotate, Scale) in the Scene view toolbar. Select a GameObject and use these tools to manipulate its position, rotation, and scale.
- Experiment with different views (2D, 3D) in the Scene view using the controls in the top-right corner.

Practice 1 – Organizing and Saving the Scene

- Rename GameObjects in the Hierarchy panel to keep the scene organized.
- Save the scene by going to File > Save Scene or use the shortcut Ctrl + S (Cmd + S on Mac).
Name the scene appropriately.

Conclusion and Next Steps

Scene Fundamentals: Explored scenes as distinct segments crucial for structuring games, emphasizing their impact on overall project flow.

Practical Techniques: Covered scene creation, navigation, organization strategies, lighting, camera setup, saving/loading scenes, Build Settings, and Player Configuration.

Hands-on Experience: Participants engaged in practical exercises, creating scenes, adding and organizing game objects, enhancing their proficiency in scene management.

References

- 1: Hocking, Joseph; Schell, Jesse, Unity in action: multiplatform game development in C#, 2022
- 3: Geig, Mike, Sams teach yourself Unity Game development in 24 hours, 2014
- 6: Gibson Bond, Jeremy, Introduction to Game Design, Prototyping, and Development: From Concept to Playable Game with Unity and C,
- 8: Unity Technologies, Unity Manual, 2023
- 10: Goldstone Will, Unity 3.x game development essentials, 2011